Are Large Language Models Good SLM Email Classifiers? A Local-First Approach to Intent Classification

Final Project Report - CS5100

1. Introduction

The proliferation of Large Language Models (LLMs) has fundamentally altered the landscape of natural language processing tasks over the past few years. From generating essays to writing complex software, their generative capabilities have received significant mainstream and academic attention. However, their foundational utility in deterministic, complex classification tasks remains an area requiring rigorous investigation. This is particularly relevant in domains demanding high privacy and low latency, such as personal email triage.

Individuals and corporations alike receive hundreds of emails daily, ranging from critical financial alerts and academic communications to redundant promotional spam. While automating the sorting and drafting process is highly desirable, the underlying mechanism involves exposing highly sensitive personal or financial information. Relying on cloud-based APIs (such as OpenAI's GPT-4 or Anthropic's Claude) necessitates sending this private data to external servers, creating unacceptable vectors for data breaches and violating strict privacy protocols. This constraint has driven the demand for "Local-First" AI architectures—systems that can intelligently operate entirely on consumer-grade local hardware without an internet connection.

This project investigates the development of a privacy-preserving, entirely local email assistant designed to fetch, analyze, and draft responses to emails by classifying their underlying intent. To inform this approach, I grounded the methodology in the findings of Ruan, Kuznetsov, and Gurevych (2024) in their recent paper *"Are Large Language Models Good Classifiers? A Study on Edit Intent Classification in Scientific Document Revisions"* (EMNLP). Ruan et al. systematically evaluated LLM performance on intent classification. They observed that while unconstrained generative models exhibit strong zero-shot reasoning capabilities, their outputs are inherently unstable. When used as classifiers, generative models require either strict structural constraints or specialized encoding-based fine-tuning to provide robust and reliable categorization.

Echoing these findings, this project addresses two core challenges of local agentic AI: maintaining absolute data privacy during processing and extracting reliable, structured intent from generative models. By employing computationally efficient Small Language Models (SLMs) running locally alongside strict structural generation constraints (via Pydantic JSON schemas) and Parameter-Efficient Fine-Tuning (PEFT) techniques, this system reliably extracts intent (categorizing emails as Actionable vs. Ignore) without reliance on external cloud servers, demonstrating a scalable approach to local digital automation.

2. Related Work

2.1 Generative Models vs. Classification Constraints

Historically, natural language classification was achieved using discriminative models (e.g., SVMs, Random Forests) trained on Bag-of-Words or TF-IDF representations. With the advent of the Transformer architecture, smaller encoder representations like BERT (Bidirectional Encoder Representations from Transformers) became the gold standard for text classification. However, the modern paradigm has shifted towards decoder-only generative models (like Llama). As Ruan et al. (2024) highlighted, forcing a decoder-only language model into a strict classification task often results in conversational "fluff" or hallucinated output formats (e.g., the model prepending "Here is your classification data:"). This project builds upon this discourse by introducing deterministic JSON scaffolding to constrain generative divergence.

2.2 Parameter-Efficient Fine-Tuning (PEFT)

To adapt a pre-trained language model to a specific task, full-parameter fine-tuning is computationally prohibitive. Hu et al. (2021) introduced Low-Rank Adaptation (LoRA), a technique that freezes the vast majority of the pre-trained model weights. Instead of training billions of parameters, LoRA injects trainable rank decomposition matrices into the attention layers of the transformer. This reduces the number of trainable parameters by over 10,000x and GPU memory requirements by 3x. For an entirely local email agent running on consumer Apple Silicon, LoRA serves as the pivotal mechanism allowing a generalized SLM (such as Qwen 2.5) to be rapidly domain-adapted specifically for email intent mapping.

---

3. Methodology

To fulfill the requirements of an autonomous but safe email assistant, the project follows a containerized microservices architecture coordinated via Docker Compose. This ensures modularity, scalability, and environmental reproducibility. The system is split into multiple distinct layers: Data Ingestion, Local Inference, Persistence, and frontend Human-in-the-Loop authorization.

3.1 Overview of Architecture

The local agent relies on a multi-tiered tech stack:

- **Backend API**: A highly asynchronous Python FastAPI server handles all the business logic, IMAP orchestration, and database routing.
- **Frontend Interface**: A Streamlit application serves as the interactive dashboard for the user to trigger scans, view intelligence metrics, and authorize drafts.
- **AI Engine**: Ollama, executing locally on the host machine's neural engine or GPU, manages the heavy lifting of the generalized SLMs (Llama 3.2).
- **Storage Layer**: PostgreSQL serves as the persistent state manager to ensure identical emails are not repeatedly analyzed.

3.2 Ingestion Module (IMAP & MIME parsing)

Before any intelligence can be applied, the system must establish a secure bridge to the user's external email provider. The FastAPI backend utilizes Python's native `imaplib` library, authenticating against Gmail via secure App Passwords to bypass multi-factor authentication loops intended for browsers. Once authenticated, the server fetches the latest raw email bytes over SSL.

Emails are inherently complex, often nested in Multipurpose Internet Mail Extensions (MIME) formats, containing varied layers of HTML, raw text, hidden tracking pixels, and attachments. The ingestion module recursively traverses the MIME tree, specifically targeting the `text/plain` content type to extract a clean string. The text is subsequently truncated to the first 500 characters, preventing heavy HTML artifacts or egregiously long threads from overwhelming the context window (and VRAM capacity) of the local SLM.

3.3 Inference Engine and Structural Constraints

A foundational technical challenge in this project was mapping the unbounded generative capability of Llama 3.2 into rigidly defined database columns. As established by Ruan et al. (2024), generative models require strict enforcement to act as classifiers.

To combat conversational drift (where the AI outputs "Sure! Happy to help, here is the JSON you requested..."), the backend incorporates **Pydantic** data validation integrated directly into Ollama's structural formatting mechanism.

```
class EmailAnalysis(BaseModel):
    category: str = Field(description="[Finance/Academic/Personal/Spam/General]")
    action: str = Field(description="[Reply/Ignore]")
    draft: str = Field(description="[If the action is Reply, provide a short,
professional draft response here. Otherwise, return an empty string]")
```

By mapping the standard definition of the `EmailAnalysis` Pydantic class to a strict JSON Schema, the Llama model executes grammar-constrained generation. This mathematically

prevents the language model from generating any tokens that violate the required schema schema, guaranteeing 100% parse success rates when loading the resulting output into Python dictionary formats and transmitting it to the PostgreSQL database.

3.4 Data Synthesis and Domain Adaptation

Zero-shot prompting, while effective, is computationally expensive due to the large token overhead required to instruct a model on classification rules. To explore a more optimized pathway, I curated a synthetic dataset consisting of ~800 typical emails and their expected JSON counterparts. The data covers distinct categories representing an average student/professional's inbox.

An excerpt of the `dataset.json` used for training:

```
 {
    "input_email": "Subject: CS5100 Final Project Extension\nFrom:
prof.smith@university.edu\nBody: Hi students, I have decided to extend the deadline
for the final project by 48 hours. Please ensure your progress reports are submitted
by Friday.",
    "output_json": {
      "category": "Academic",
      "action": "Reply",
      "draft": "Thank you for the update, Professor. I will make sure to submit the
progress report by the new Friday deadline."
    }
  }
```

Using the Hugging Face `trl` and `peft` libraries, a lightweight base model (Qwen 2.5 0.5B) underwent Supervised Fine-Tuning (SFT) over 3 epochs. By employing LoRA configurations targeting the `q_proj` and `v_proj` attention layers, the localized model mathematically shifted its weights to organically output the targeted categories ("Finance, Academic, Spam") without needing verbose zero-shot instruction prepended to the prompt.

3.5 Human-in-the-Loop UI Design

Automated agents that possess write-permissions directly to an external API (such as sending emails on a user's behalf) pose high operational risks, especially if the local model hallucinates a draft. To safely implement autonomous emailing, I engineered a "Human-in-the-Loop" workflow inside the Streamlit frontend.

Rather than instantly sending an outbound response when an email is flagged as "Actionable," the Streamlit dashboard visually aggregates the database. The user can select specific records, view the LLM's suggested draft in an interactive `st.text_area`,

manually edit any tone discrepancies, and only then press a definitive "Approve & Send" button, executing the outbound SMTP routing from the FastAPI backend.

---

4. Evaluation and Results

To systematically evaluate the performance of this system—and to empirically validate the differences between instructed generation and encoded structural learning—an evaluation script (`evaluate.py`) was executed over an unseen subset of 20 emails. The script recorded both processing latency and categorical mapping accuracy against the expected manual labels.

4.1 Quantitative Benchmarks

Llama 3.2 (Zero-Shot via Ollama):

- **Average Latency:** ~3.01 seconds/email
- **Schema Strictness (Formatting):** 100%
- **Raw Categorical Output Match:** 0.0%

Qwen 0.5B (LoRA Fine-Tuned):

- **Average Latency:** ~0.94 seconds/email
- **Schema Strictness (Formatting):** 100%
- **Category / Action Accuracy (Intent Match):** >95.0%

4.2 Qualitative Interpretations

The benchmark results offer compelling evidence supporting the theoretical assertions made earlier in the literature.

Zero-Shot Discrepancies: The most immediate observation is the effective 0% category match rate for the Zero-shot Llama model. This failure is not a testament to the model's inability to analyze text. Rather, without hundreds of tokens explicitly defining what "categories" are allowed, the generalized model simply inferred its own arbitrary, logical categories (e.g., "School Update" instead of the rigid database enum "Academic"). It struggled to rigidly map intent to the tight constraints of the hardcoded system without exhaustive prompt engineering.

LoRA Triumphs: By contrast, the much smaller Qwen 0.5B adapter naturally learned the exact JSON structural casing, precisely targeted the five acceptable categories, and perfectly determined the "Reply/Ignore" binary directly through its PEFT weights. It confirmed the observations by Ruan et al. (2024) that fine-tuning fundamentally transforms generative decoders into superlative intent classifiers.

Latency Improvements: The fine-tuned SLM also executed ~3x faster (under one second per email) compared to the zero-shot prompting of Llama 3.2. By eliminating the necessity for heavy system instructions in the input sequence, the matrix multiplication overhead was significantly reduced, paving the way for real-time inbox synchronization loops.

---

5. Discussion

The system successfully navigates the complex problem space of autonomous email agents while completely sidestepping cloud-infrastructure reliance. Implementing structural dependencies like Pydantic proved absolutely critical for building an interface between unstructured neural text and structured SQL databases.

The "Local-First" methodology shines in its economic and privacy guarantees. Operating locally via Ollama means the operational cost scales with the host machine's electricity, rather than OpenAI's per-token billing metric. More importantly, personal academic discussions, financial notices, and personal correspondence are processed entirely in memory and on local storage, establishing an absolute fortress against external telemetry or data harvesting.

---

6. Conclusion

This project successfully proves the viability of a "Local-First" AI agent for personal email triage, fulfilling the requirements for high data-privacy and operational utility. By addressing the core problems of structural inference highlighted in modern NLP literature, the architecture establishes a deterministic, automated workflow that effectively triages and drafts contextual responses.

6.1 Strengths

1. **Absolute Data Privacy**: No raw text arrays or classified contexts are transmitted over external APIs. Processing is contained entirely on the developer machine.

2. **Deterministic Output Pipelines**: Implementing grammar-enforced Pydantic constraints ensures that any connected backend application can safely deserialize the LLM output directly into database records without crashing.

3. **Workflow Safety**: The Streamlit interface's explicit "Review & Edit" functionality prevents autonomous failure cascades, maintaining user sovereignty over outbox permissions.

6.2 Weaknesses & Future Work

Despite the successful implementation, local processing bears inherent bottlenecks.

1. **Context Window Limitations**: Small local models often possess limited context windows (typically 4k-8k tokens) and are bottlenecked by local device RAM. Extremely long, recursive email chains laden with historical replies may cause Out-Of-Memory (OOM) errors or truncation degradation, severing vital context for draft generation.

2. **Inverted Speed/Scale Mechanics**: Execution throughput is completely bounded by the parallel processing capacity of local consumer GPUs or NPUs (Apple Silicon). Cloud APIs can theoretically scale processing in parallel; here, processing massive backlog inboxes sequentially requires significant patience.

Future iterations of this agent could radically mitigate these weaknesses by integrating a Retrieval-Augmented Generation (RAG) framework over the PostgreSQL database. Implementing a local vector store (such as ChromaDB) would allow the agent to fetch historical emails across different threads, utilizing long-term context when drafting a reply without needing the entire history loaded into the primary LLM prompt.

---

7. Works Cited

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.
- Ruan, Q., Kuznetsov, I., & Gurevych, I. (2024). Are Large Language Models Good Classifiers? A Study on Edit Intent Classification in Scientific Document Revisions. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Hugging Face. (2024). *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning*. https://github.com/huggingface/peft
- FastAPI. (2024). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. https://fastapi.tiangolo.com/
- Streamlit. (2024). *The fastest way to build custom ML tools*. https://streamlit.io/

- Ollama. (2024). *Get up and running with Llama 3, Mistral, Gemma 2, and other large language models.* https://ollama.com/